테이블 시스템 개요

목적

테이블 시스템은 ZKEVM의 여러 회로들 간에 데이터를 공유하고 검증하는 핵심 메커니즘입니다. 각 회로는 독립적으로 동작하지만, 테이블을 통해 다른 회로가 생성한 데이터를 참조하고 검증할 수 있습니다. 이를 통해 회로 간 일관성을 보장하고, 복잡한 EVM 실행을 여러 특화된 회로로 분산하여 검증할 수 있습니다.

회로 간 데이터 공유 메커니즘

Lookup 기반 검증

테이블 시스템의 핵심은 **Lookup** 메커니즘입니다. 한 회로가 테이블에 데이터를 기록(write)하면, 다른 회로는 그 테이블을 조회(lookup)하여 데이터의 존재와 정확성을 검증합니다.

```
┌──────────────┐  ┌──────────────┐  ┌──────────────┐

| EVM Circuit| ──────>| RW Table |<────── |State Circuit|

| (Lookup) | | (Shared) | | (Write) |

└──────────────┘  └──────────────┘  └──────────────┘
```

동작 원리:

1. **Write**: 한 회로가 테이블에 데이터를 할당(assign)합니다
2. **Lookup**: 다른 회로가 해당 테이블을 조회하여 데이터가 존재하는지 검증합니다
3. **Constraint**: Halo2의 lookup 제약 조건을 통해 조회한 값이 테이블에 실제로 존재함을 증명합니다

LookupTable 트레이트

모든 테이블은 `LookupTable<F>` 트레이트를 구현합니다:

파일: `zkevm-circuits/src/table.rs`

```rust
pub trait LookupTable<F: Field> {
/// 테이블의 모든 컬럼 반환
fn columns(&self) -> Vec<Column<Any>>;
/// Advice 컬럼만 반환
fn advice_columns(&self) -> Vec<Column<Advice>>;
/// Fixed 컬럼만 반환
fn fixed_columns(&self) -> Vec<Column<Fixed>>;
/// 컬럼 주석 반환
fn annotations(&self) -> Vec<String>;
/// Lookup에 사용되는 Expression 반환
fn table_exprs(&self, meta: &mut VirtualCells<F>) -> Vec<Expression<F>>;
}
```

이 트레이트를 통해 각 테이블은 일관된 인터페이스를 제공하며, 회로에서 쉽게 lookup을 수행할 수 있습니다.

테이블 타입 개요

ZKEVM 구현체는 다양한 목적의 테이블을 제공합니다. 각 테이블은 특정 유형의 데이터를 저장하고 공유합니다.

1. 핵심 실행 테이블

RwTable (Read-Write Table)

- **목적**: EVM 실행 중 발생하는 모든 읽기/쓰기 연산 추적
- **사용 회로**: EVM Circuit (lookup), State Circuit (write)
- **주요 컬럼**: `rw_counter`, `is_write`, `tag`, `id`, `address`, `value`, `value_prev`
- **파일**: `zkevm-circuits/src/table.rs` (line 617)

```rust
pub struct RwTable {
pub q_enable: Column<Fixed>,
pub rw_counter: Column<Advice>,
pub is_write: Column<Advice>,
pub tag: Column<Advice>,
pub id: Column<Advice>,
pub address: Column<Advice>,
pub field_tag: Column<Advice>,
pub storage_key: Column<Advice>,
pub value: Column<Advice>,
pub value_prev: Column<Advice>,
pub aux1: Column<Advice>,
pub aux2: Column<Advice>,
}
```

CopyTable

- **목적**: 메모리, 바이트코드, 트랜잭션 데이터 간 복사 연산 검증

- **사용 회로**: Copy Circuit (write), EVM Circuit (lookup)
- **주요 컬럼**: `is_first`, `id`, `addr`, `src_addr_end`, `bytes_left`, `rlc_acc`
- **파일**: `zkevm-circuits/src/table.rs` (line 1728)

2. 트랜잭션 및 블록 테이블

TxTable (Transaction Table)

- **목적**: 트랜잭션의 모든 필드 저장
- **사용 회로**: Tx Circuit (write), EVM Circuit (lookup)
- **주요 컬럼**: `tx_id`, `tag`, `index`, `value`, `access_list_address`
- **파일**: `zkevm-circuits/src/table.rs` (line 212)

```
pub struct TxTable {
pub q_enable: Column<Fixed>,
pub tx_id: Column<Advice>,
pub tag: Column<Advice>,
pub index: Column<Advice>,
pub value: Column<Advice>,
pub access_list_address: Column<Advice>,
pub chunk_txbytes_hash_rlc: Column<Advice>,
}
```

BlockTable

- **목적**: 블록 헤더 필드 저장
- **사용 회로**: EVM Circuit (lookup)

- **주요 컬럼**: `tag`, `index`, `value`
- **파일**: `zkevm-circuits/src/table.rs` (line 1349)

3. 코드 및 해시 테이블

BytecodeTable

- **목적**: 스마트 컨트랙트 바이트코드 저장 및 검증
- **사용 회로**: Bytecode Circuit (write), EVM Circuit (lookup)
- **주요 컬럼**: `code_hash`, `tag`, `index`, `is_code`, `value`, `push_rlc`
- **파일**: `zkevm-circuits/src/table.rs` (line 1186)

KeccakTable

- **목적**: Keccak256 해시 연산 결과 검증
- **사용 회로**: Keccak Circuit (write), 여러 회로 (lookup)
- **주요 컬럼**: `is_final`, `input_rlc`, `input_len`, `output_rlc`
- **파일**: `zkevm-circuits/src/table.rs` (line 1439)

```
pub struct KeccakTable {
pub q_enable: Column<Fixed>,
pub is_final: Column<Advice>,
pub input_rlc: Column<Advice>,
pub input_len: Column<Advice>,
pub output_rlc: Column<Advice>,
}
```

PoseidonTable

- **목적**: Poseidon 해시 연산 (zkTrie용)
- **사용 회로**: Hash Circuit, MPT Circuit, Bytecode Circuit
- **주요 컬럼**: `hash_id`, `input0`, `input1`, `control`, `domain_spec`, `heading_mark`
- **파일**: `zkevm-circuits/src/table.rs` (line 947)

SHA256Table

- **목적**: SHA256 프리컴파일 해시 검증
- **사용 회로**: SHA256 Circuit (write), EVM Circuit (lookup)
- **주요 컬럼**: `is_final`, `input_rlc`, `input_len`, `output_rlc`
- **파일**: `zkevm-circuits/src/table.rs` (line 1606)

4. 상태 및 증명 테이블

MptTable (Merkle Patricia Trie Table)

- **목적**: 이더리움 상태 트리 업데이트 검증
- **사용 회로**: MPT Circuit (write), State Circuit (lookup)
- **주요 컬럼**: `address`, `storage_key`, `proof_type`, `new_root`, `old_root`, `new_value`, `old_value`
- **파일**: `zkevm-circuits/src/table.rs` (line 788)

```
pub struct MptTable {
pub q_enable: Column<Fixed>,
pub address: Column<Advice>,
pub storage_key: Column<Advice>,
pub proof_type: Column<Advice>,
pub new_root: Column<Advice>,
```

```
pub old_root: Column<Advice>,
pub new_value: Column<Advice>,
pub old_value: Column<Advice>,
}
```

5. 암호화 및 프리컴파일 테이블

SigTable (Signature Table)

- **목적**: ECDSA 서명 검증 (secp256k1, secp256r1)
- **사용 회로**: Sig Circuit (write), Tx Circuit, EVM Circuit (lookup)
- **주요 컬럼**: `msg_hash_rlc`, `sig_v`, `sig_r_rlc`, `sig_s_rlc`, `recovered_addr`, `is_valid`
- **파일**: `zkevm-circuits/src/table.rs` (line 2550)

EccTable (Elliptic Curve Table)

- **목적**: 타원곡선 연산 (EcAdd, EcMul, EcPairing) 검증
- **사용 회로**: ECC Circuit (write), EVM Circuit (lookup)
- **파일**: `zkevm-circuits/src/table.rs` (line 2764)

ModExpTable (Modular Exponentiation Table)

- **목적**: 모듈러 지수 연산 프리컴파일 검증
- **사용 회로**: ModExp Circuit (write), EVM Circuit (lookup)
- **파일**: `zkevm-circuits/src/table.rs` (line 2963)

6. 특수 목적 테이블

ExpTable (Exponentiation Table)

- **목적**: EXP opcode 연산 검증
- **사용 회로**: Exp Circuit (write), EVM Circuit (lookup)
- **주요 컬럼**: `is_step`, `is_last`, `base_limb`, `exponent_lo_hi`, `exponentiation_lo_hi`
- **파일**: `zkevm-circuits/src/table.rs` (line 2180)

RlpFsmRlpTable

- **목적**: RLP 인코딩 검증 (트랜잭션용)
- **사용 회로**: RLP Circuit (write), Tx Circuit (lookup)
- **주요 컬럼**: `tx_id`, `format`, `rlp_tag`, `tag_value`, `tag_bytes_rlc`, `tag_length`
- **파일**: `zkevm-circuits/src/table.rs` (line 2383)

7. 유틸리티 테이블

PowOfRandTable

- **목적**: Keccak randomness의 거듭제곱 값 저장 (최적화용)
- **주요 컬럼**: `exponent`, `pow_of_rand`
- **파일**: `zkevm-circuits/src/table.rs` (line 3148)

RangeTable

- **목적**: [0, MAX) 범위 검증
- **사용**: 값이 특정 범위 내에 있는지 확인
- **파일**: `zkevm-circuits/src/table.rs` (line 3284)

Pow2Table

- **목적**: 2의 거듭제곱 값 저장

- **사용**: 비트 연산 및 범위 검증
- **파일**: `zkevm-circuits/src/table.rs` (line 3327)

BitwiseOpTable

- **목적**: 비트 단위 AND/OR/XOR 연산 결과 저장
- **사용**: 비트 연산 검증
- **파일**: `zkevm-circuits/src/table.rs` (line 3396)

테이블 간 관계

테이블들은 서로 연결되어 전체 EVM 실행의 일관성을 보장합니다:

```
┌──────────────┐
| TxTable |──┐
└──────────────┘ |
├──> ┌──────────────┐ ┌──────────────┐
┌──────────────┐ | | EVM Circuit | ──> | RwTable |
| BytecodeTable|──┤ | (Execution) | | |
└──────────────┘ | └──────────────┘ └──────────────┘
| | |
┌──────────────┐ | | |
| KeccakTable |──┘ v v
└──────────────┘ ┌──────────────┐ ┌──────────────┐
| CopyTable | |State Circuit |
└──────────────┘ └──────────────┘
```

```
|
v
┌──────────────┐
| MptTable |
└──────────────┘
```

주요 데이터 흐름:

1. **트랜잭션 입력**: TxTable → EVM Circuit
2. **코드 실행**: BytecodeTable → EVM Circuit
3. **상태 변경**: EVM Circuit → RwTable → State Circuit
4. **해시 검증**: 여러 회로 → KeccakTable/PoseidonTable
5. **상태 증명**: State Circuit → MptTable
6. **데이터 복사**: EVM Circuit → CopyTable

RLC (Random Linear Combination) 사용

많은 테이블에서 **RLC**를 사용하여 여러 바이트를 하나의 필드 원소로 압축합니다:

```
// 예: KeccakTable에서 input_rlc 계산
input_rlc = challenges.keccak_input().map(|challenge|
rlc::value(input.iter().rev(), challenge)
);
```

RLC의 장점:

- 여러 컬럼을 하나로 압축하여 회로 크기 감소
- 바이트 배열을 효율적으로 비교 가능
- Challenge를 사용하여 보안성 확보

테이블 할당 패턴

테이블 데이터는 일반적으로 다음 패턴으로 할당됩니다:

```
// 1. 테이블 생성
let table = MyTable::construct(meta);

// 2. 데이터 할당 (dev_load 또는 load 메서드)
table.dev_load(layouter, data, challenges)?;

// 3. 다른 회로에서 lookup
meta.lookup_any("lookup name", |meta| {
let table_exprs = table.table_exprs(meta);
// ... lookup 제약 조건 정의
});
```

성능 최적화

테이블 시스템은 다음과 같은 최적화 기법을 사용합니다:

1. **SecondPhase 컬럼**: RLC 값은 SecondPhase에 할당하여 challenge 사용

2. **Fixed 컬럼**: 변하지 않는 값(태그 등)은 Fixed 컬럼 사용

3. **q_enable**: 각 행의 활성화 여부를 제어하여 불필요한 제약 조건 방지

4. **패딩**: 고정된 테이블 크기를 위해 빈 행을 패딩으로 채움

다음 단계

각 테이블의 상세한 구조와 사용 방법은 다음 문서에서 다룹니다:

- 02-RW-Table.md - Read-Write 테이블 상세
- 03-Bytecode-Table.md - 바이트코드 테이블 상세
- 04-Tx-Table.md - 트랜잭션 테이블 상세
- 05-Block-Table.md - 블록 테이블 상세
- 06-Copy-Table.md - 복사 테이블 상세
- 07-Keccak-Table.md - Keccak 테이블 상세
- 08-MPT-Table.md - MPT 테이블 상세
- 09-Lookup-메커니즘.md - Lookup 동작 원리 상세

참고 자료

- **코드**: `zkevm-circuits/src/table.rs`
- **관련 문서**:
- EVM Circuit
- State Circuit
- RLC 기법